

■ GRACEemo ViRa

Master ROS 2 Hands-On Tutorial & Self-Paced Lab Guide

Institutional Project: School of Computer Science & Engineering | Lovely Professional University

Mentor: Dr. Mohit Arora | **Project Lead:** Sam Davi | **Target Distro:** ROS 2 Jazzy (Ubuntu 24.04)

Lesson 0: Environment & Daily Workflow

Your ROS 2 environment runs inside the Linux Docker container `ros2_learn` with direct live volume mapping to your workspace folder.

The Golden 2-Step Routine for ANY Terminal:

```
# Step 1: Jump into the active ROS 2 container
docker exec -it ros2_learn bash

# Step 2: Source the ROS 2 setup environment
source /opt/ros/jazzy/setup.bash
```

Lesson 1: Nodes & Topics (Publish / Subscribe)

In ROS 2, a **Node** is an isolated executable performing a specific function. Nodes communicate asynchronously over named channels called **Topics**.

1.1 Building a Telemetry Publisher:

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32, String
import random

class TelemetryPub(Node):
    def __init__(self):
        super().__init__('telemetry_publisher')
        self.batt_pub = self.create_publisher(Float32, '/graceemo/battery_voltage', 10)
        self.timer = self.create_timer(0.5, self.publish_telemetry)
        self.voltage = 12.6

    def publish_telemetry(self):
        self.voltage -= random.uniform(0.001, 0.005)
        msg = Float32()
        msg.data = round(self.voltage, 3)
        self.batt_pub.publish(msg)
        self.get_logger().info(f'■ Voltage: {msg.data}V')

def main():
    rclpy.init()
    node = TelemetryPub()
    try:
        rclpy.spin(node)
    except:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok(): rclpy.shutdown()

if __name__ == '__main__': main()
```

1.2 Building a Battery Safety Watchdog Subscriber:

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32

class BatteryWatchdog(Node):
    def __init__(self):
        super().__init__('battery_watchdog')
        self.sub = self.create_subscription(
            Float32, '/graceemo/battery_voltage', self.callback, 10)

    def callback(self, msg):
        if msg.data < 12.0:
            self.get_logger().error(f'■ CRITICAL VOLTAGE: {msg.data}V! Triggering Safe E-Stop!')
        else:
            self.get_logger().info(f'■ Voltage Normal: {msg.data}V')

def main():
    rclpy.init()
    node = BatteryWatchdog()
    try:
        rclpy.spin(node)
    except:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok(): rclpy.shutdown()

if __name__ == '__main__': main()
```

Lesson 2: Command-Line Power Tools (Inspection & Debugging)

Essential CLI tools to inspect the active ROS 2 computation graph in real time:

Command	Description
ros2 node list	List all currently running nodes
ros2 node info <node_name>	Show publishers, subscribers, services of a node
ros2 topic list -t	List all active topics along with their message types
ros2 topic echo <topic>	Stream live data published on a topic to terminal
ros2 topic hz <topic>	Measure live publish rate (frequency in Hz)
ros2 topic pub --once <topic> <type> '<data>'	Manually broadcast a single message from the terminal

Lesson 3: Services (Request ■ Response Architecture)

While topics provide continuous streams, **Services** provide synchronous, two-way Remote Procedure Calls (client requests a task, server computes and returns a response).

```
# Service Server Example
from example_interfaces.srv import Trigger
import rclpy
from rclpy.node import Node

class GraceEMOService(Node):
    def __init__(self):
        super().__init__('graceemo_status_service')
        self.srv = self.create_service(
            Trigger, '/graceemo/get_campus_status', self.handle_request)
        self.get_logger().info('■■ Campus Status Service Online')

    def handle_request(self, request, response):
        response.success = True
        response.message = "LPU Campus: 600 Acres | NAAC A++ | All Systems Operational"
        return response

def main():
    rclpy.init()
    node = GraceEMOService()
    try: rclpy.spin(node)
    finally: node.destroy_node(); rclpy.shutdown()

if __name__ == '__main__': main()
```

Lesson 4: Custom Message & Service Definitions

Custom schemas are defined inside a dedicated messages package (e.g. graceemo_msgs):

```
# PersonDetection.msg (Message definition)
int32 track_id
string name
float32 confidence
geometry_msgs/Point position_in_map

---
# IdentifyPerson.srv (Service definition with Request and Response)
sensor_msgs/Image face_crop
---
bool is_authorized
string full_name
string designation
```

Lesson 5: Actions (Long-Running Tasks with Feedback)

Actions are used for preemptible tasks that take measurable time (e.g., navigating to a room, executing a speech dialogue, or executing a motion trajectory).

Component	Function
Goal	Client sends target destination or command (e.g. target: 'Library')
Feedback	Server continuously transmits incremental progress (e.g. distance_remaining: 1.2m)
Result	Server returns final status upon completion (e.g. arrived: True, duration: 32s)
Preemption	Client can cancel the goal at any time while in progress

Lesson 6: Node Parameters & Config Files (YAML)

Never hardcode values. Parameters allow external configuration at startup or runtime dynamic tuning.

```
# Declaring and using parameters in Python
self.declare_parameter('max_speed', 0.5)
self.declare_parameter('robot_name', 'GraceEMO_V1')

# Reading parameter value
current_speed = self.get_parameter('max_speed').get_parameter_value().double_value

# Dynamic CLI inspection and live modification:
# ros2 param list
# ros2 param get /node_name max_speed
# ros2 param set /node_name max_speed 1.2 <-- Instant live update!
```

Lesson 7: Workspaces, Packages & Build System (colcon)

ROS 2 uses colcon to build multiple modular packages in a clean workspace tree:

```
# Workspace Directory Structure:
# graceemo_ws/
#   src/ (source packages)
#   build/ (compilation artifacts)
#   install/ (deployed binaries and setup.bash)

# 1. Create a new Python package
cd ~/graceemo_ws/src
ros2 pkg create --build-type ament_python graceemo_brain --dependencies rclpy std_msgs geometry_msgs

# 2. Build the workspace
cd ~/graceemo_ws
colcon build --symlink-install

# 3. Source the built workspace
source install/setup.bash
```

Lesson 8: Launch Files (Multi-Node Orchestration)

Launch files allow you to start your entire robot stack (perception, navigation, voice, and drivers) with a single command: `ros2 launch graceemo_bringup robot.launch.py`.

```
# Python Launch File Example (robot.launch.py)
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='graceemo_perception',
            executable='detector_node',
            name='yolo_detector',
            parameters=[{'confidence_thresh': 0.5, 'device': 'cuda:0'}]
        ),
        Node(
            package='graceemo_voice',
            executable='stt_node',
            name='speech_transcriber'
        )
    ])

```

Lesson 9: TF2 Transform System (3D Spatial Awareness)

TF2 maintains the coordinate tree representing the relative 3D position and orientation of every sensor and link:

Coordinate Tree:

map (World) ■■■ *odom (Odometry)* ■■■ *base_link (Robot Center)* ■■■ *camera_link / lidar_link*

Lesson 10: Building an AI Vision Node (YOLO + ROS 2)

Bridge OpenCV image arrays and ROS 2 standard image messages seamlessly:

```

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge

class VisionNode(Node):
    def __init__(self):
        super().__init__('vision_node')
        self.bridge = CvBridge()
        self.sub = self.create_subscription(
            Image, '/camera/image_raw', self.image_callback, 10)

    def image_callback(self, msg):
        # Convert ROS Image -> OpenCV BGR Numpy Array
        cv_frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')

        # Run YOLO / AI Inference here:
        # detections = yolo_model(cv_frame)
        self.get_logger().info(f'Processed Frame: {cv_frame.shape}')

def main():
    rclpy.init(); node = VisionNode()
    try: rclpy.spin(node)
    finally: node.destroy_node(); rclpy.shutdown()

```

Master Troubleshooting Cheatsheet

Error / Symptom	Root Cause	Immediate Fix
ModuleNotFoundError: No module named 'rclpy'	Executed on host macOS instead of container	Run docker exec -it ros2_learn bash first
ros2: command not found	ROS 2 environment not sourced	Run source /opt/ros/jazzy/setup.bash
Nodes cannot discover each other	Domain ID mismatch across shells	Run export ROS_DOMAIN_ID=0 in all terminals
Service call hangs indefinitely	Target service server is not active	Check with ros2 service list